

PARACASYA_PROMPTS_IMPORTANTES.md

Prompts reutilizables para Codex

Proyecto: ParacasYa Market Online

1. Propósito del documento

Este documento reúne los prompts más importantes para seguir trabajando en **ParacasYa Market Online** desde una nueva cuenta de ChatGPT o ChatGPT Pro de empresa.

La idea es que Walter pueda copiar y pegar estos prompts en Codex para:

- Auditar el proyecto.
- Trabajar nuevas fases.
- Agregar productos.
- Validar cambios.
- Hacer commit/push controlado.
- Proteger secretos.
- Evitar romper funcionalidades estables.

2. Prompt base universal para cualquier fase

Usar este prompt como base cada vez que se empiece una fase nueva.

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

[NOMBRE DE LA FASE]

Objetivo:

[EXPLICAR OBJETIVO CONCRETO]

Contexto actual:

- La app ya tiene /cliente, /caja y /admin funcionando.
- Los productos se leen desde Supabase.
- Los pedidos se crean desde /cliente.
- Los pedidos aparecen en /caja.
- Telegram avisa cuando entra un pedido.

- WhatsApp funciona desde caja.
- QR/origen hotel funciona con ?origen=...
- Admin usa Edge Function admin-catalog para catálogo.
- No asumir que Cloudinary ya sube imágenes automáticamente; actualmente se pega image_url manualmente.

Restricciones:

- No tocar .env ni .env.local.
- No imprimir secrets, tokens ni claves reales.
- No tocar RLS.
- No tocar Edge Functions salvo que la fase lo pida explícitamente.
- No tocar pedidos, caja, WhatsApp, Telegram, QR/origen hotel ni seguimiento salvo que la fase lo pida.
- No abrir escrituras anónimas.
- No agregar DELETE.
- No tocar Supabase remoto salvo que se indique explícitamente.
- No hacer commit ni push todavía.

Antes de modificar:

1. Ejecutar git status --short.
2. Confirmar que el working tree esté limpio o explicar cambios pendientes.
3. Revisar archivos relevantes.

Validación obligatoria:

1. Ejecutar npm run build.
2. Ejecutar git status --short.
3. Revisar git diff.
4. Confirmar archivos modificados.
5. Confirmar áreas sensibles no tocadas.

Entrega final:

- Qué se hizo.
- Archivos modificados.
- Build pasó/falló.
- Riesgos.
- Pruebas manuales recomendadas.
- Confirmación de no tocar .env, secrets, RLS, Edge Functions, pedidos, caja, WhatsApp, Telegram ni QR si no correspondía.
- No hacer commit ni push.

3. Prompt de auditoría técnica real

Este prompt sirve para pedirle a Codex una radiografía del proyecto sin modificar nada.

Trabajemos en ParacasYa Market.

Carpeta del proyecto:

C:\Users\Usuario\Documents\ParacasYa Market Online

Objetivo:

Crear un resumen técnico real y completo del proyecto ParacasYa Market para migrarlo a otra cuenta de ChatGPT / ChatGPT Pro de empresa.

IMPORTANTE:

Esta tarea es solo de análisis y documentación.

No modificar archivos.

No aplicar migraciones.

No tocar Supabase remoto.

No tocar Vercel.

No tocar Cloudinary.

No tocar Telegram.

No tocar WhatsApp.

No tocar Edge Functions.

No tocar .env ni .env.local.

No imprimir secrets, tokens, claves reales ni valores sensibles.

No hacer commit.

No hacer push.

Necesito que analices el proyecto actual y generes un informe técnico claro en formato Markdown.

El informe debe incluir:

1. Estado general del proyecto:

- Nombre del proyecto.
- Carpeta local.
- Rama Git actual.
- Último commit local.
- Estado de git status --short.
- Si hay archivos sin trackear o modificados.
- Si el working tree está limpio o no.

2. Stack técnico:

- Framework frontend.
- Build tool.
- Librerías principales detectadas en package.json.
- Scripts disponibles.
- Comando para correr local.
- Comando para build.
- Comando para preview si existe.

3. Estructura de carpetas relevante:

- src/
- src/pages/
- src/components/
- src/services/
- src/utils/
- src/data/
- src/styles/
- supabase/
- supabase/functions/

4. Rutas principales:

- /
- /cliente
- /caja
- /admin
- seguimiento de pedido si existe
- rutas de QR/origen hotel si existen

5. Supabase:

- archivo donde se crea el cliente Supabase,
- nombres de variables de entorno usadas, sin imprimir valores,
- tablas detectadas,
- operaciones principales por tabla,
- Edge Functions,
- migraciones SQL.

6. Tablas:

- products
- categories
- orders
- order_items
- store_settings
- cualquier otra tabla relevante.

7. Productos y catálogo:

- cómo se cargan productos,
- cómo se filtran categorías,
- cómo se manejan disponibles/destacados,
- cómo se guarda image_url,
- estado del catálogo de cervezas.

8. Cloudinary:

- si hay subida real,
- si solo se pega image_url,
- variables detectadas,
- riesgos.

9. Pedidos:

- flujo técnico completo,
- orders,
- order_items,
- createOrder,
- Telegram.

10. WhatsApp:

- normalización,
- wa.me,
- mensajes.

11. QR/origen hotel:

- query param,
- localStorage,
- order_origin,
- order_origin_label,
- generador QR.

12. Seguimiento:

- componente,
- polling,
- estados.

13. Admin:

- productos,
- categorías,
- tienda abierta/cerrada,
- admin-catalog.

14. Caja:

- listado,
- estados,
- WhatsApp,
- métricas,
- Realtime/polling/sonido si existe.

15. Edge Functions:

- admin-catalog
- notify-order-telegram
- cualquier otra.

16. Variables de entorno:

Crear tabla con:

- Variable
- Dónde se usa
- Para qué sirve
- Sensibilidad

- Local
- Vercel
- Supabase Function

No imprimir valores reales.

17. Seguridad:

- puntos correctos,
- puntos sensibles,
- qué no tocar.

18. Estado de Fase 3D:

- evidencias en /cliente,
- mejoras aplicadas,
- riesgos.

19. Build:

Ejecutar npm run build y documentar resultado.

20. Git:

Ejecutar:

git status --short

git log --oneline -5

21. Riesgos actuales.

22. Recomendaciones técnicas inmediatas.

23. Resumen ejecutivo final.

Formato:

Markdown claro y ordenado.

Recordatorio:

No modificar archivos.

No imprimir secrets.

No hacer commit.

No hacer push.

4. Prompt para agregar productos por SQL seguro

Usar este prompt cuando se quieran agregar productos nuevos al catálogo desde Supabase SQL.

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

Agregar productos nuevos al catálogo

Objetivo:

Agregar productos al catálogo de ParacasYa Market de forma segura, sin borrar ni reemplazar productos existentes.

Contexto:

- Los productos se guardan en Supabase.
- Las tablas principales son products y categories.
- /cliente muestra productos con is_available = true.
- /admin permite gestionar productos.
- No se deben duplicar productos.
- No se debe tocar pedidos, caja, WhatsApp, Telegram ni QR.

Productos a agregar:

[PEGAR AQUÍ LISTA DE PRODUCTOS CON NOMBRE, PRECIO, CATEGORÍA, DESCRIPCIÓN Y SI ES DESTACADO]

Tarea técnica:

1. Revisar estructura actual de products y categories.
2. Revisar SQL existentes en supabase/.
3. Crear una migración SQL local segura en supabase/, por ejemplo:
supabase/paracasya_[nombre]_catalog.sql

La migración debe:

- Crear la categoría solo si no existe.
- Reutilizar categorías existentes si corresponden.
- Insertar productos sin duplicar.
- Usar is_available = true.
- Usar is_featured según corresponda.
- Usar stock_status compatible con el proyecto, por ejemplo available.
- Usar image_url = null si no hay imagen.
- No borrar productos.
- No modificar RLS.
- No abrir escrituras públicas.
- No agregar DELETE.
- No tocar orders ni order_items.

Validación:

1. Ejecutar npm run build.
2. Revisar git diff.

3. Confirmar archivo SQL creado.
4. Confirmar si se aplicó o no a Supabase remoto.
5. No hacer commit ni push todavía.

Entrega final:

- archivo creado,
- productos agregados,
- categoría usada,
- si se aplicó a Supabase remoto,
- build,
- áreas no tocadas.

5. Prompt específico: catálogo de cervezas

Este prompt ya se usó para agregar el catálogo de cervezas y puede servir como referencia.

Trabajemos en ParacasYa Market.

Carpeta del proyecto:

C:\Users\Usuario\Documents\ParacasYa Market Online

Objetivo:

Agregar nuevos productos de cervezas a la plataforma, sumándolos al catálogo actual sin borrar ni reemplazar productos existentes.

Contexto:

- ParacasYa Market es el kiosco online de Paracas.
- La plataforma ya tiene /cliente, /admin y /caja funcionando.
- Los productos se leen desde Supabase.
- El admin ya permite crear y editar productos mediante la Edge Function admin-catalog.
- El proyecto usa categorías y productos.
- Los productos nuevos deben aparecer en /cliente como disponibles.
- No tocar pedidos, caja, WhatsApp, Telegram, QR/origen hotel, seguimiento de pedido, store_settings ni lógica existente.
- No tocar .env, .env.local, secrets, tokens ni variables de Vercel.
- No abrir escrituras anon.
- No modificar RLS de forma insegura.
- No agregar DELETE.
- No hacer commit ni push todavía.

Productos a agregar en la categoría:

Bebidas alcohólicas

Si la categoría "Bebidas alcohólicas" no existe, crearla.
Si ya existe una categoría similar como "Alcohol", usar la existente para no duplicar innecesariamente.
Los productos deben quedar activos/disponibles en tienda.

Lista de productos:

1. Cerveza Pilsen lata 355 ml

Precio: S/7.00

Descripción: Cerveza Pilsen lata 355 ml. Venta solo para mayores de 18 años.
Tomar bebidas alcohólicas en exceso es dañino.

Destacado: no

2. Cerveza Cusqueña lata/botella

Precio: S/8.50

Descripción: Cerveza Cusqueña lata o botella. Venta solo para mayores de 18 años. Tomar bebidas alcohólicas en exceso es dañino.

Destacado: no

3. Six pack Cusqueña

Precio: S/42.00

Descripción: Six pack de cerveza Cusqueña. Ideal para compartir. Venta solo para mayores de 18 años. Tomar bebidas alcohólicas en exceso es dañino.

Destacado: sí

4. Six pack Pilsen lata 355 ml

Precio: S/35.00

Descripción: Six pack de cerveza Pilsen lata 355 ml. Ideal para grupos, hoteles y Airbnbs. Venta solo para mayores de 18 años. Tomar bebidas alcohólicas en exceso es dañino.

Destacado: sí

5. Corona botella 355 ml

Precio: S/11.00

Descripción: Cerveza Corona botella 355 ml. Opción premium para turistas. Venta solo para mayores de 18 años. Tomar bebidas alcohólicas en exceso es dañino.

Destacado: no

6. Six pack Corona botella 355 ml

Precio: S/60.00

Descripción: Six pack de cerveza Corona botella 355 ml. Opción premium para compartir. Venta solo para mayores de 18 años. Tomar bebidas alcohólicas en exceso es dañino.

Destacado: sí

7. Tres Cruces lata/botella

Precio: S/7.50

Descripción: Cerveza Tres Cruces lata o botella. Venta solo para mayores de 18

años. Tomar bebidas alcohólicas en exceso es dañino.

Destacado: no

8. Stella Artois botella/lata

Precio: S/10.00

Descripción: Cerveza Stella Artois botella o lata. Opción premium. Venta solo para mayores de 18 años. Tomar bebidas alcohólicas en exceso es dañino.

Destacado: no

Tarea técnica:

1. Revisar el esquema actual de Supabase para categories y products.
2. Revisar si ya existe alguna migración o seed de productos.
3. Crear una migración SQL local segura:
supabase/paracasya_beer_catalog.sql
4. La migración debe:
 - Insertar la categoría "Bebidas alcohólicas" solo si no existe, o reutilizar "Alcohol" si ya existe.
 - Insertar los 8 productos sin duplicar.
 - Si un producto ya existe por nombre, no duplicarlo.
 - Usar image_url como null por ahora.
 - Usar is_available = true.
 - Usar is_featured según la lista.
 - Usar stock_status compatible con el proyecto actual, por ejemplo "available".
 - No borrar productos.
 - No modificar productos existentes, salvo que sea necesario para evitar duplicados.
 - No tocar pedidos, órdenes ni otras tablas.
5. Si es seguro aplicar el SQL al Supabase remoto desde la CLI ya conectada, hacerlo con mucho cuidado.
6. Si no se puede aplicar remoto sin credenciales o sin confirmar conexión, dejar el SQL listo y explicar cómo aplicarlo manualmente desde Supabase SQL Editor.
7. Ejecutar npm run build.
8. Revisar git diff.
9. Confirmar:
 - Archivos creados/modificados.
 - Productos agregados.
 - Categoría usada.
 - Si se aplicó o no a Supabase remoto.
 - Que no se tocaron .env, .env.local, secrets, RLS inseguro, pedidos, caja, WhatsApp, Telegram, QR ni seguimiento.
10. No hacer commit ni push todavía.

6. Prompt para commit y push controlado

Usar solo cuando Walter ya haya revisado el cambio y quiera subirlo a GitHub.

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Objetivo:

Hacer commit y push controlado de los cambios ya implementados y validados.

Antes de hacer commit:

1. Ejecutar `git status --short`.
2. Ejecutar `git diff`.
3. Ejecutar `npm run build`.
4. Confirmar que no hay `.env`, `.env.local`, `secrets`, `tokens` ni archivos sensibles.
5. Confirmar que los archivos modificados corresponden solo a la fase trabajada.

Commit:

Usar este mensaje:

[MENSAJE DE COMMIT]

Luego:

```
git add [archivos específicos]
git commit -m "[MENSAJE DE COMMIT]"
git push origin main
```

Restricciones:

- No agregar `.env`.
- No agregar `.env.local`.
- No subir `secrets`.
- No hacer cambios nuevos antes del commit.
- No tocar `RLS`.
- No tocar `Supabase` remoto.
- No modificar `Edge Functions` salvo que ya hayan sido parte validada de la fase.
- No incluir archivos no relacionados.

Entrega final:

- Commit creado.
- Hash del commit.
- Push realizado.
- Archivos incluidos.
- Build aprobado.
- `git status --short` final.

7. Prompt para prueba operativa real de producción

Este debe ser el primer prompt/prueba después de migrar el proyecto.

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

Prueba operativa real de producción

Objetivo:

Validar de punta a punta que el flujo actual funciona correctamente en producción y local, sin modificar código.

IMPORTANTE:

Solo análisis y pruebas.

No modificar archivos.

No tocar Supabase remoto salvo lectura.

No tocar .env ni .env.local.

No imprimir secrets.

No hacer commit.

No hacer push.

Flujo a validar:

cliente → pedido → Supabase → caja → Telegram → WhatsApp → seguimiento

Pruebas:

1. Revisar que el proyecto compile:

npm run build

2. Confirmar rutas existentes:

- /
- /cliente
- /caja
- /admin

3. Probar URL con origen QR:

/cliente?origen=hotel-paracas

4. Crear un pedido de prueba con productos reales.

Usar datos de prueba claros:

Nombre: Prueba ParacasYa

WhatsApp: número de prueba autorizado por Walter

Ubicación: Hotel Paracas Prueba
Referencia: Habitación 101
Comentarios: Pedido de prueba, no entregar
Método de pago: Efectivo

5. Validar que el pedido:
 - aparece en Supabase,
 - aparece en /caja,
 - conserva order_origin y order_origin_label,
 - llega a Telegram,
 - permite abrir WhatsApp,
 - muestra seguimiento en el recibo.
6. Cambiar estados desde /caja:
 - Pendiente
 - Confirmado
 - Preparando
 - En camino
 - Entregado
7. Validar que el recibo refleja cambios.

Entrega final:

- Qué funcionó.
- Qué falló.
- Capturas o descripción si aplica.
- Estado de Telegram.
- Estado de WhatsApp.
- Estado de seguimiento.
- Recomendaciones.
- Confirmar que no se modificó código.

8. Prompt para Fase 3A — Catálogo nocturno completo

Trabajemos en ParacasYa Market.

Carpeta:
C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:
Fase 3A — Catálogo nocturno completo

Objetivo:
Agregar productos básicos nocturnos que los turistas, hoteles, Airbnbs y

residentes pueden necesitar de noche.

Contexto:

- Ya existe catálogo de cervezas.
- Ya existe categoría Bebidas alcohólicas.
- Los productos se leen desde Supabase.
- /cliente muestra productos disponibles.
- /admin permite gestionar productos.
- No se debe romper ningún flujo actual.

Productos sugeridos:

[PEGAR LISTA FINAL APROBADA POR WALTER]

Reglas:

- Agregar productos sin duplicar.
- Reutilizar categorías existentes si corresponde.
- Crear categorías solo si no existen.
- No modificar productos existentes salvo que se indique.
- image_url = null si no hay imagen.
- is_available = true.
- stock_status = available.
- is_featured = true solo para productos clave.
- No tocar RLS.
- No tocar pedidos.
- No tocar /caja.
- No tocar WhatsApp.
- No tocar Telegram.
- No tocar QR.
- No tocar Edge Functions.
- No hacer commit ni push.

Validación:

1. Crear SQL local seguro en supabase/.
2. Aplicar remoto solo si está confirmado y es seguro.
3. Ejecutar npm run build.
4. Revisar git diff.
5. Confirmar productos/categorías agregadas.
6. Confirmar que no hay duplicados.
7. No hacer commit ni push.

Entrega final:

- archivo SQL creado,
- productos agregados,
- categorías usadas,
- si se aplicó a Supabase remoto,
- build,
- áreas no tocadas.

9. Prompt para Fase 3B — Combos comerciales

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

Fase 3B – Combos comerciales

Objetivo:

Agregar combos comerciales como productos independientes para subir el ticket promedio.

Contexto:

- Los combos se tratarán como productos normales.
- No implementar lógica de pack ni descuento automático en esta fase.
- No tocar stock.
- No tocar pedidos.
- No tocar caja.
- No tocar backend sensible.

Combos a crear:

[PEGAR LISTA FINAL APROBADA POR WALTER]

Ejemplos:

- Combo Noche: Coca-Cola 1.5 L + papas + chocolate – S/18
- Combo Cervezas: Six pack Pilsen + hielo – S/42
- Combo Airbnb: Agua grande + gaseosa + snacks – S/25
- Combo Bajón: Gaseosa personal + papas + galleta – S/13

Tarea:

1. Revisar categoría Combos o Promos.
2. Crear categoría si no existe.
3. Insertar combos como productos independientes.
4. Marcar combos principales como destacados.
5. image_url = null por ahora.
6. is_available = true.
7. stock_status = available.

Restricciones:

- No modificar productos base.
- No tocar orders ni order_items.
- No tocar RLS.
- No tocar Edge Functions.

- No tocar Telegram.
- No tocar WhatsApp.
- No hacer commit ni push.

Validación:

- npm run build
- git diff
- confirmar productos en Supabase si se aplicó
- no duplicados

Entrega final:

- combos agregados,
- categoría usada,
- SQL creado,
- si se aplicó remoto,
- build,
- áreas no tocadas.

10. Prompt para Fase 3C — Imágenes principales manuales

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

Fase 3C – Imágenes principales manuales

Objetivo:

Actualizar productos principales con image_url de Cloudinary pegadas manualmente desde /admin o SQL seguro.

Contexto:

- Cloudinary no está integrado como subida automática.
- Actualmente /admin permite pegar image_url.
- No hay variables Cloudinary.
- No agregar upload automático en esta fase.

Productos prioritarios:

[PEGAR LISTA DE PRODUCTOS Y URLS CLOUDINARY]

Tarea:

1. Revisar productos existentes.
2. Actualizar solo image_url de productos indicados.

3. No modificar nombre, precio ni disponibilidad salvo que se indique.
4. Si se usa SQL, crear archivo local seguro.
5. No exponer secretos.
6. No crear integración Cloudinary.

Restricciones:

- No tocar Edge Functions.
- No tocar .env.
- No tocar secrets.
- No tocar RLS.
- No tocar pedidos.
- No tocar caja.
- No tocar Telegram.
- No tocar WhatsApp.
- No hacer commit ni push.

Validación:

- Confirmar imágenes en /cliente.
- Confirmar fallback si imagen falla.
- npm run build.
- git diff.

Entrega final:

- productos actualizados,
- URLs agregadas sin secretos,
- método usado,
- build,
- áreas no tocadas.

11. Prompt para Fase 3G — Mejoras de caja

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

Fase 3G — Mejoras de caja

Objetivo:

Mejorar /caja para operación real con actualización automática, filtros y mejor visibilidad, sin romper estados ni WhatsApp.

Contexto:

- /caja ya lista pedidos reales.
- Permite cambiar estados.
- Muestra métricas básicas.
- Muestra origen QR.
- Abre WhatsApp.
- Actualmente no hay Realtime, polling automático ni sonido detectado.

Mejoras solicitadas:

1. Agregar polling automático cada 15 segundos.
2. Mantener botón manual "Actualizar".
3. Mostrar "Última actualización: HH:mm".
4. Agregar filtros por estado:
 - Todos
 - Pendiente
 - Confirmado
 - Preparando
 - En camino
 - Entregado
 - Cancelado
5. Agregar alerta visual cuando entra un pedido nuevo.
6. Sonido opcional solo si ya hay patrón simple y no rompe UX.

Restricciones:

- No tocar orders schema.
- No tocar order_items schema.
- No tocar Telegram.
- No tocar WhatsApp.
- No tocar Edge Functions.
- No tocar Supabase RLS.
- No cambiar nombres de estados.
- No hacer commit ni push.

Validación:

1. npm run build.
2. Probar /caja.
3. Confirmar que cambiar estados funciona.
4. Confirmar que WhatsApp sigue funcionando.
5. Confirmar que filtros no ocultan pedidos incorrectamente.
6. git diff.

Entrega final:

- archivos modificados,
- cambios aplicados,
- build,
- pruebas recomendadas,
- áreas no tocadas.

12. Prompt para Fase 3H — Reporte simple de ventas

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

Fase 3H — Reporte simple de ventas

Objetivo:

Agregar un reporte simple de ventas para medir el negocio sin crear un sistema complejo.

Contexto:

- Los pedidos están en orders.
- Los items están en order_items.
- /admin ya existe.
- /caja ya muestra algunas métricas básicas.
- No borrar ni modificar pedidos.

Métricas deseadas:

- Pedidos de hoy.
- Ventas de hoy.
- Ticket promedio de hoy.
- Pedidos pendientes.
- Pedidos entregados.
- Pedidos cancelados.
- Total semanal.
- Producto más vendido si se puede calcular sin complejidad.

Tarea:

1. Revisar si conviene agregar sección en /admin o /caja.
2. Crear funciones de lectura si hace falta.
3. Leer orders y order_items.
4. Calcular métricas en frontend.
5. No crear tablas nuevas en esta fase salvo que sea necesario.
6. No modificar esquema sin autorización.

Restricciones:

- No tocar RLS.
- No tocar Edge Functions.
- No tocar creación de pedidos.
- No tocar Telegram.
- No tocar WhatsApp.
- No borrar pedidos.

- No hacer commit ni push.

Validación:

- npm run build
- probar reporte con pedidos existentes
- verificar que no rompe /admin ni /caja
- git diff

Entrega final:

- ubicación del reporte,
- métricas agregadas,
- archivos modificados,
- build,
- áreas no tocadas.

13. Prompt para Fase 3E — Zonas de delivery

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Fase:

Fase 3E — Zonas de delivery

Objetivo:

Agregar zonas de delivery para ordenar la operación y permitir tarifas según ubicación.

Contexto:

- Actualmente el cliente escribe hotel/Airbnb/dirección.
- orders tiene location_name y delivery_fee.
- No romper pedidos existentes.
- No tocar caja más de lo necesario.

Zonas sugeridas:

- Centro de Paracas — S/5
- Malecón — S/5
- Zona hoteles — S/8
- Airbnb / hospedaje — S/8
- Otro — coordinar

Tarea:

1. Revisar esquema orders.

2. Proponer si conviene agregar campos nuevos o reutilizar delivery_notes/location_name.
3. Si se agrega campo, crear SQL local seguro.
4. Agregar selector de zona en checkout.
5. Calcular delivery_fee según zona.
6. Mostrar zona y delivery en /caja.
7. Mantener compatibilidad con pedidos anteriores.

Restricciones:

- No tocar RLS sin autorización.
- No borrar pedidos.
- No modificar order_items.
- No tocar Telegram salvo que se deba incluir zona en mensaje y sea seguro.
- No tocar WhatsApp salvo mensaje mínimo.
- No hacer commit ni push.

Validación:

- npm run build.
- crear pedido de prueba.
- verificar delivery_fee.
- verificar /caja.
- git diff.

Entrega final:

- cambios propuestos/aplicados,
- SQL si existe,
- archivos modificados,
- build,
- pruebas pendientes.

14. Prompt para revisar seguridad sin cambiar nada

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Objetivo:

Auditar seguridad del proyecto sin modificar nada.

IMPORTANTE:

Solo lectura.

No modificar archivos.

No tocar Supabase remoto.

No cambiar RLS.
No tocar Edge Functions.
No imprimir secrets.
No hacer commit.
No hacer push.

Revisar:

1. Si hay service_role en frontend.
2. Si .env o .env.local están trackeados.
3. Si hay tokens impresos en logs.
4. Si hay DELETE implementado.
5. Si hay escrituras anon inseguras en SQL local.
6. Si Edge Functions validan tokens.
7. Si admin usa admin-catalog y no anon directo para escribir catálogo.
8. Si pedidos usan anon de manera controlada.
9. Si Telegram expone secretos.
10. Si variables VITE son usadas correctamente.

Entregar:

- puntos correctos,
- riesgos,
- severidad alta/media/baja,
- recomendaciones,
- archivos a revisar,
- cambios sugeridos para una fase futura.

No modificar nada.

15. Prompt para cierre de fase con resumen

Usar cuando Codex ya terminó una fase y queremos que entregue resumen claro.

Necesito un resumen final de la fase aplicada en ParacasYa Market.

Formato obligatorio:

Fase:

[Nombre de la fase]

Estado:

Completada / Parcial / Fallida

Archivos modificados:

- ...

Archivos creados:

- ...

Qué se hizo:

- ...

Validación:

- npm run build: pasó/falló
- git status --short:
- git diff revisado: sí/no

Pruebas manuales recomendadas:

- ...

No se tocó:

- .env
- .env.local
- secrets
- RLS
- Edge Functions
- Supabase remoto
- pedidos
- /caja
- WhatsApp
- Telegram
- QR/origen hotel
- seguimiento de pedido

Riesgos:

- ...

Recomendación:

- hacer commit / no hacer commit todavía

No hacer commit ni push salvo que yo lo pida.

16. Prompt para crear documento Markdown dentro del repo

Usar si se quiere guardar estos documentos dentro del proyecto.

Trabajemos en ParacasYa Market.

Carpeta:

C:\Users\Usuario\Documents\ParacasYa Market Online

Objetivo:

Crear un documento Markdown dentro del repo para documentación del proyecto.

Nombre del archivo:

docs/[NOMBRE_DEL_DOCUMENTO].md

Contenido:

[PEGAR CONTENIDO DEL DOCUMENTO]

Restricciones:

- Crear carpeta docs/ si no existe.
- No modificar código fuente.
- No tocar Supabase.
- No tocar .env ni .env.local.
- No tocar secrets.
- No hacer commit ni push todavía.

Validación:

1. git status --short.
2. Confirmar archivo creado.
3. No hace falta npm run build si solo se creó documentación, salvo que se haya tocado código.

Entrega final:

- archivo creado,
- ruta,
- confirmación de que no se tocó código,
- git status.

17. Prompt para migración a nueva cuenta ChatGPT

Usar al iniciar el proyecto en la nueva cuenta.

Este proyecto se llama ParacasYa Market Online.

Necesito que actúes como asistente técnico y estratégico del proyecto.

Contexto:

ParacasYa Market es el kiosco online de Paracas: una tienda local, rápida y confiable para pedir bebidas, snacks, hielo, cervezas, combos y productos básicos sin salir del hotel, Airbnb o casa.

Stack:

- React + Vite.
- Supabase.
- Vercel.
- GitHub.
- Edge Functions.
- Telegram.
- WhatsApp.
- QR por hotel/Airbnb.

Rutas:

- /
- /cliente
- /caja
- /admin

Estado:

MVP avanzado funcionando.

Último commit documentado:

90ea544 feat: agregar catalogo de cervezas

Reglas:

- Trabajar por fases.
- No tocar .env ni .env.local.
- No imprimir secrets.
- No tocar RLS sin autorización.
- No tocar Edge Functions sin autorización.
- No abrir escrituras anónimas.
- No agregar DELETE.
- No tocar pedidos/caja/WhatsApp/Telegram/QR salvo que la fase lo pida.
- No hacer commit ni push salvo que Walter lo pida.
- Siempre pedir build y git status al cerrar fase.
- Responder en español claro, práctico y directo.

Documentos cargados:

- Documento Maestro.
- Resumen Técnico Real.
- Reglas Codex.
- Roadmap Actualizado.
- Prompts Importantes.

Primera tarea:

Ayúdame a retomar el proyecto desde el roadmap y trabajar la próxima fase de forma controlada.

18. Orden recomendado de uso de estos prompts

1. Usar **Prompt de migración a nueva cuenta ChatGPT**.
2. Subir o pegar:
3. Documento Maestro.
4. Resumen Técnico Real.
5. Reglas Codex.
6. Roadmap Actualizado.
7. Prompts Importantes.
8. Usar **Prompt de auditoría técnica real** si se quiere confirmar estado.
9. Usar **Prueba operativa real de producción**.
10. Continuar con:
11. Fase 3A catálogo nocturno completo.
12. Fase 3B combos.
13. Fase 3C imágenes.
14. Fase 3G caja.
15. Fase 3H reportes.

19. Regla final

Antes de cualquier cambio importante:

Primero proteger lo que ya funciona.

ParacasYa Market ya tiene una base real. La prioridad es no romper pedidos, caja, Telegram, WhatsApp, Supabase ni QR.